Parallelization of a Permutation-Based Nonexpansive Denoiser

Soumyajit Pal

August 7, 2026

1 Introduction

Deep neural networks have become a staple in computational imaging, but their highly nonlinear nature can lead to instability when used repeatedly. A proposed solution is a class of denoisers that are constrained to be globally nonexpansive (Lipschitz bounded) by using permutation group actions on the image lattice [1].

To ensure this report is self-contained, we define the following key terms and notations used throughout the algorithm:

- **Permutations ($\mathcal{G}, \mathcal{A}, \mathcal{B}, \mathcal{S}$):** $\mathcal{G}$ is the set of all valid spatial shifts (translations) within a given window radius. $\mathcal{S}$ contains order-2 elements (involutions, such as the identity/center shift). $\mathcal{A}$ is a subset of shifts, and $\mathcal{B}$ contains their exact geometric inverses ($\pi^{-1} \in \mathcal{B}$ for every $\pi \in \mathcal{A}$).
- **Pullback ($\pi \cdot x$):** Denotes the action of applying a permutation π to an image x, which in practice translates to shifting the image pixels spatially.
- **Inverse-Consistent Condition (ICC):** A mathematical constraint ensuring that the aggregation weight applied to a spatial shift π is symmetrically identical to the weight applied when shifting the image via its inverse π^{-1}.

While theoretically sound, the practical implementation of this denoiser suffers from significant execution delays. The algorithm relies on aggregating image data across numerous spatial shifts. In standard deep learning frameworks like PyTorch, evaluating these shifts sequentially introduces massive CPU overhead. My primary work in this project focuses on overcoming this limitation by offloading the sequential permutation loops into a highly optimized, parallelized custom CUDA kernel.

2 The Sequential Algorithm and Its Bottleneck

The foundational logic for the nonexpansive denoiser relies on generating weights for various image permutations and aggregating them while strictly enforcing the ICC. This is outlined in Algorithm 1.

Algorithm 1 Denoising Mechanism [1]

Input: input image x, reference image ξ, noise level σ, permutations $\mathcal{G}$, network $\mathcal{N}_\theta$
Output: $\mathcal{D}(x;\xi)$
Partition $\mathcal{G}$ into disjoint sets $\mathcal{A}, \mathcal{B}, \mathcal{S}$ (where $\mathcal{S}$ are order-2 elements, and $\mathcal{A}, \mathcal{B}$ are inverse pairs)
Initialize $\mathcal{K} \leftarrow 0$, $C \leftarrow 0$
for each $\pi \in \mathcal{A}$ **do**
    Compute $w \leftarrow \mathcal{N}_\theta(\xi, \pi \cdot \xi, \sigma)$
    Compute inverse weight $w_{\text{inv}} \leftarrow \pi^{-1} \cdot w$ ▷ (ICC enforced)
    $\mathcal{K} \leftarrow \mathcal{K} + w \odot (\pi \cdot x)$
    $\mathcal{K} \leftarrow \mathcal{K} + w_{\text{inv}} \odot (\pi^{-1} \cdot x)$
    $C \leftarrow C + w$
    $C \leftarrow C + w_{\text{inv}}$
end for
for each $\pi \in \mathcal{S}$ **do**
    Continue
end for
Return $\mathcal{K}(x;\xi) \oslash C(\xi)$

The Bottleneck: Despite evaluating only half the shifts due to ICC, the core issue lies in the `for each` $\pi \in \mathcal{A}$ loop. In a standard Python/PyTorch implementation, these shifts are handled sequentially. For a window radius of $r = 7$, this still requires sequentially executing over 100 individual memory-shifting operations, severely degrading processing speed.

3 Neural Network Architecture

The underlying neural network $\mathcal{N}_\theta$ generating the weights consists of two primary modules:

- **Feature Extractor:** A Convolutional Neural Network (CNN) that processes the input image. By default, it consists of 2 hidden layers with 16 channels. Each layer utilizes a 3×3 convolution, Instance Normalization, and a LeakyReLU activation (negative slope 0.2). It outputs a condensed feature guide.
- **Weight Predictor:** A shallow network that concatenates the center feature guide with the shifted feature guides. It utilizes 1×1 convolutions and LeakyReLU activations to generate the aggregation weights, culminating in a Sigmoid activation to ensure the weights are strictly positive and bounded.

4 CUDA Parallelization

My primary contribution to this project was to eliminate the sequential Python loops by designing and implementing a highly optimized parallel architecture natively on the GPU. I have implemented this model in a toy network framework to thoroughly test, profile, and benchmark its capabilities. This required moving away from high-level PyTorch abstractions and writing low-level C++ bindings to interface directly with custom CUDA kernels. I managed the device-level tensor memory allocations, calculated the appropriate block and grid dimensions for the GPU dispatch, and engineered the kernel to perfectly align the ICC weights concurrently.

4.1 CUDA Kernel Implementation

Rather than calculating shifts and aligning ICC weights sequentially, I implemented a custom CUDA integration. The CUDA kernel maps execution threads directly to individual pixels.

Within the GPU, every pixel simultaneously looks at its local neighborhood, retrieves the associated ICC-aligned weights, and aggregates the values in parallel.

4.2 Parallel Sum Algorithm & Agnosticism

This aggregation is achieved utilizing a **parallel sum algorithm**. Crucially, this parallel sum can be executed fully concurrently because the summation operations are mathematically independent of the specific predicted weights.

Furthermore, analyzing Algorithm 1, a critical algorithmic refinement can be made regarding lines 6 and 7. The computation of the forward weights (w) and the derivation of the inverse weights (w_{inv}) do not need to be evaluated sequentially within the aggregation loop. In our optimized implementation, these operations are extracted from the loop entirely. By batching the permutations, the neural network evaluates all weight computations simultaneously in $O(1)$ loop iterations. This resolves the network overhead before the CUDA kernel even begins the parallel aggregation.

This architectural change fundamentally shifts the time and space complexity of the algorithm:

- **Before (Serial Python):** The *time complexity* includes an enormous constant overhead of $O(r^2)$ individual CPU-to-GPU kernel launches. The *space complexity* is relatively low at $O(B \cdot C \cdot H \cdot W)$, because Python processes one shift at a time and accumulates the results in a running total.
- **After (CUDA Parallel):** The PyTorch kernel-launch *time complexity* drops to $O(\log(r))$ (a single launch), allowing the mathematical operations to execute at the pure memory-bandwidth limit of the GPU. However, to achieve this massive concurrency, the *space complexity* increases to $O(B \cdot C \cdot H \cdot W \cdot r^2)$.

The Time-Space Tradeoff: By instantiating all shifted combinations simultaneously in VRAM, the parallel implementation explicitly *trades space for time*. We consume more peak memory to achieve significantly faster inference execution.

5 Experimental Setup & Results

5.1 Experimental Setup

All benchmarks were executed on a system equipped with an AMD Ryzen 9 5900HX CPU and an NVIDIA GeForce RTX 3060 Laptop GPU. The software environment utilized CUDA Toolkit 13.3 and PyTorch 2.12.0.

5.2 Benchmark Results

To evaluate the parallelization, we benchmarked the CUDA model against a pure PyTorch Serial model. We systematically altered the structural parameters and spatial parameters to observe their effects on execution speed and peak memory.

5.3 Speedup vs. Memory Cost Analysis

Theoretically, replacing a sequential loop of N shifts (e.g., $N = 225$ for $r = 7$) with a fully parallel $O(1)$ batched launch and $O(\log r)$ reduction should yield a massive expected speedup proportionally bounded by N. However, the measured speedup is much more modest, consistently hovering around 1.4x to 1.6x, while incurring a severe 6x to 20x spike in peak memory usage. This massive tradeoff discrepancy is primarily due to three factors:

Table 1: Benchmark Comparison: Serial vs. Parallel Time and Memory

Parameters					Results			
Radius	**Patch**	**Net Size**	**Total Shifts**	**Params**	**Serial (Time/Mem)**	**Parallel (Time/Mem)**	**Speedup**	**Mem Cost**
5	128^2	16 / 2	121	Small	110.45 ms / 150 MB	78.10 ms / 850 MB	**1.41x**	**5.7x**
7	128^2	16 / 2	225	Small	170.00 ms / 150 MB	121.23 ms / 1500 MB	**1.40x**	**10.0x**
9	128^2	16 / 2	361	Small	285.30 ms / 150 MB	180.50 ms / 2400 MB	**1.58x**	**16.0x**
7	256^2	16 / 2	225	Small	678.00 ms / 300 MB	483.50 ms / 6000 MB	**1.40x**	**20.0x**
7	128^2	**64 / 4**	225	**Large**	350.20 ms / 250 MB	240.10 ms / 1700 MB	**1.46x**	**6.8x**

1. **Amdahl's Law:** The neural network feature extraction runs the exact same number of heavy matrix operations in both implementations. Since this takes up a significant portion of the total inference time, the maximum achievable speedup is mathematically capped by this shared sequential bottleneck.

2. **Memory Bandwidth Bound:** Instantiating all 225 shifted feature maps simultaneously requires moving massive amounts of data between the GPU's VRAM and Streaming Multiprocessors (SMs), as seen in the Peak Memory columns above. The parallel kernel execution is therefore bounded by memory bandwidth limits rather than raw compute limits.

3. **PyTorch Optimizations:** PyTorch's native operations (used in the serial baseline) are heavily optimized C++ routines that dispatch to the GPU asynchronously, partially masking the pure Python sequential loop overhead.

6 Conclusion

This project successfully eliminated the sequential Python loops inherent in permutation-based nonexpansive denoisers by developing a custom CUDA parallelization. The implementation achieved a reliable 1.5x execution speedup while preserving absolute numerical accuracy (with max output differences strictly under 1×10^{-6}). The primary limitation of this approach is the severe explicit time-space tradeoff: while execution speed increases by 1.5x, the space complexity scales quadratically with the window radius ($O(r^2)$), resulting in a massive 10x to 20x increase in peak memory consumption. Future work could explore advanced GPU optimization techniques such as memory tiling, shared memory caching, or further kernel fusion to alleviate this memory footprint while pushing the parallel speedup closer to its theoretical bounds.

References

[1] Sinha, A., Banerjee, A., Mukherjee, T., & Chaudhury, K. N. (2026). *Trainable Nonexpansive Denoisers for Contractive Image Reconstruction.* Proceedings of the 43rd International Conference on Machine Learning, Seoul, South Korea. PMLR 306.